

Namespace RepositorioRemoto.Api

Interfaces

[IJsonPlaceholderApi](#)

Cliente tipado de la API REST de JSONPlaceholder. Refit genera la implementación en tiempo de ejecución a partir de los atributos [Get], [Post], [Put] y [Delete]: no hay que escribir ni una línea de HttpClient.

Interface IJsonPlaceholderApi

Namespace: RepositorioRemoto.[Api](#)

Assembly: RepositorioRemoto.dll

Cliente tipado de la API REST de JSONPlaceholder. Refit genera la implementación en tiempo de ejecución a partir de los atributos [Get], [Post], [Put] y [Delete]: no hay que escribir ni una línea de HttpClient.

```
[Headers(new string[] { "Content-Type: application/json" })]  
public interface IJsonPlaceholderApi
```

Methods

CreateUserAsync(CreateUserRequest)

Crea un usuario. La API devuelve el usuario con el id que le asigna.

```
[Post("/users")]  
Task<JsonPlaceholderUserDto> CreateUserAsync(CreateUserRequest request)
```

Parameters

request [CreateUserRequest](#)

Datos del usuario a crear.

Returns

[Task](#) [<JsonPlaceholderUserDto>](#)

DeleteUserAsync(int)

Elimina un usuario.

```
[Delete("/users/{id}")]
```

Task `DeleteUserAsync(int id)`

Parameters

`id` [int](#)

Identificador del usuario.

Returns

[Task](#)

GetUserByIdAsync(int)

Obtiene un usuario por su identificador.

```
[Get("/users/{id}")]
```

```
Task<JsonPlaceholderUserDto> GetUserByIdAsync(int id)
```

Parameters

`id` [int](#)

Identificador del usuario.

Returns

[Task](#) [<JsonPlaceholderUserDto>](#)

GetUsersAsync()

Obtiene todos los usuarios.

```
[Get("/users")]
```

```
Task<List<JsonPlaceholderUserDto>> GetUsersAsync()
```

Returns

[Task](#) <[List](#) <[JsonPlaceholderUserDto](#)>>

UpdateUserAsync(int, UpdateUserRequest)

Actualiza un usuario existente.

```
[Put("/users/{id}")]
```

```
Task<JsonPlaceholderUserDto> UpdateUserAsync(int id, UpdateUserRequest request)
```

Parameters

id [int](#)

Identificador del usuario.

request [UpdateUserRequest](#)

Datos nuevos.

Returns

[Task](#) <[JsonPlaceholderUserDto](#)>

Namespace RepositorioRemoto.Cache

Classes

[MemoryCacheService](#)

Implementación de ICacheService con la caché en memoria del proceso.

[RedisCacheService](#)

Implementación de ICacheService con Redis. Es la del perfil Prod: la caché vive fuera del proceso y se puede compartir entre varias instancias.

Interfaces

[ICacheService](#)

Contrato de la caché. Lo implementan MemoryCacheService (perfil Dev) y RedisCacheService (perfil Prod).

Interface ICacheService

Namespace: RepositorioRemoto.[Cache](#)

Assembly: RepositorioRemoto.dll

Contrato de la caché. Lo implementan MemoryCacheService (perfil Dev) y RedisCacheService (perfil Prod).

```
public interface ICacheService
```

Methods

ClearAsync()

Vacía la caché entera. Lo usan el arranque y la sincronización.

```
Task ClearAsync()
```

Returns

[Task](#)

GetAsync<T>(string)

Obtiene un valor de la caché, o null si no está.

```
Task<T?> GetAsync<T>(string key) where T : class
```

Parameters

key [string](#)

Clave del valor, por ejemplo "user:1".

Returns

[Task](#) <T>

Type Parameters

T

RemoveAsync(string)

Borra un valor de la caché. Devuelve false si no estaba.

```
Task<bool> RemoveAsync(string key)
```

Parameters

key [string](#)

Clave del valor.

Returns

[Task](#) <[bool](#)>

SetAsync<T>(string, T, TimeSpan?)

Guarda un valor en la caché.

```
Task SetAsync<T>(string key, T value, TimeSpan? expiration = null) where T : class
```

Parameters

key [string](#)

Clave del valor.

value T

Valor a guardar.

expiration [TimeSpan](#)?

Tiempo que vive en la caché. Si es null, se usa el de la configuración.

Returns

[Task](#)

Type Parameters

T

Class MemoryCacheService

Namespace: RepositorioRemoto.[Cache](#)

Assembly: RepositorioRemoto.dll

Implementación de ICacheService con la caché en memoria del proceso.

```
public class MemoryCacheService : ICacheService
```

Inheritance

[object](#) ← MemoryCacheService

Implements

[ICacheService](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

MemoryCacheService(IMemoryCache, IOptions<CacheConfig>)

Implementación de ICacheService con la caché en memoria del proceso.

```
public MemoryCacheService(IMemoryCache cache, IOptions<CacheConfig> options)
```

Parameters

cache [IMemoryCache](#)

options [IOptions](#) <[CacheConfig](#)>

Methods

ClearAsync()

Vacía la caché entera. Lo usan el arranque y la sincronización.

```
public Task ClearAsync()
```

Returns

[Task](#)

GetAsync<T>(string)

Obtiene un valor de la caché, o null si no está.

```
public Task<T?> GetAsync<T>(string key) where T : class
```

Parameters

key [string](#)

Clave del valor, por ejemplo "user:1".

Returns

[Task](#) <T>

Type Parameters

T

RemoveAsync(string)

Borra un valor de la caché. Devuelve false si no estaba.

```
public Task<bool> RemoveAsync(string key)
```

Parameters

key [string](#)

Clave del valor.

Returns

[Task](#) <[bool](#)>

SetAsync<T>(string, T, TimeSpan?)

Guarda un valor en la caché.

```
public Task SetAsync<T>(string key, T value, TimeSpan? expiration = null) where T : class
```

Parameters

key [string](#)

Clave del valor.

value T

Valor a guardar.

expiration [TimeSpan](#)?

Tiempo que vive en la caché. Si es null, se usa el de la configuración.

Returns

[Task](#)

Type Parameters

T

Class RedisCacheService

Namespace: RepositorioRemoto.[Cache](#)

Assembly: RepositorioRemoto.dll

Implementación de ICacheService con Redis. Es la del perfil Prod: la caché vive fuera del proceso y se puede compartir entre varias instancias.

```
public class RedisCacheService : ICacheService
```

Inheritance

[object](#) ← RedisCacheService

Implements

[ICacheService](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

RedisCacheService(IConnectionMultiplexer,
IOptions<CacheConfig>)

Implementación de ICacheService con Redis. Es la del perfil Prod: la caché vive fuera del proceso y se puede compartir entre varias instancias.

```
public RedisCacheService(IConnectionMultiplexer redis, IOptions<CacheConfig> options)
```

Parameters

redis IConnectionMultiplexer

options [IOptions](#) <[CacheConfig](#)>

Methods

ClearAsync()

Vacía la caché entera. Lo usan el arranque y la sincronización.

```
public Task ClearAsync()
```

Returns

[Task](#)

GetAsync<T>(string)

Obtiene un valor de la caché, o null si no está.

```
public Task<T?> GetAsync<T>(string key) where T : class
```

Parameters

key [string](#)

Clave del valor, por ejemplo "user:1".

Returns

[Task](#) <T>

Type Parameters

T

RemoveAsync(string)

Borra un valor de la caché. Devuelve false si no estaba.

```
public Task<bool> RemoveAsync(string key)
```

Parameters

key [string](#)

Clave del valor.

Returns

[Task](#) <[bool](#)>

SetAsync<T>(string, T, TimeSpan?)

Guarda un valor en la caché.

```
public Task SetAsync<T>(string key, T value, TimeSpan? expiration = null) where T : class
```

Parameters

key [string](#)

Clave del valor.

value T

Valor a guardar.

expiration [TimeSpan](#)?

Tiempo que vive en la caché. Si es null, se usa el de la configuración.

Returns

[Task](#)

Type Parameters

T

Namespace RepositorioRemoto.Config

Classes

[ApiConfig](#)

Configuración de la API REST remota (JSONPlaceholder).

[CacheConfig](#)

Configuración de la caché.

[DatabaseConfig](#)

Configuración de la base de datos local.

[ExportConfig](#)

Configuración de la exportación de usuarios a JSON.

[SyncConfig](#)

Configuración de la sincronización periódica con la API remota.

Class ApiConfig

Namespace: RepositorioRemoto.[Config](#)

Assembly: RepositorioRemoto.dll

Configuración de la API REST remota (JSONPlaceholder).

```
public class ApiConfig
```

Inheritance

[object](#)  ← ApiConfig

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Fields

SectionName

Nombre de la sección en appsettings.json.

```
public const string SectionName = "ApiSettings"
```

Field Value

[string](#) 

Properties

BaseUrl

URL de la API REST.

```
public string BaseUrl { get; set; }
```

Property Value

[string](#) 

Class CacheConfig

Namespace: RepositorioRemoto.[Config](#)

Assembly: RepositorioRemoto.dll

Configuración de la caché.

```
public class CacheConfig
```

Inheritance

[object](#)  ← CacheConfig

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Fields

SectionName

Nombre de la sección en appsettings.json.

```
public const string SectionName = "CacheSettings"
```

Field Value

[string](#) 

Properties

ExpirationSeconds

Segundos que un usuario permanece en la caché.

```
public int ExpirationSeconds { get; set; }
```

Property Value

[int](#)

Provider

Tipo de caché: "Memory" (perfil Dev) o "Redis" (perfil Prod).

```
public string Provider { get; set; }
```

Property Value

[string](#)

RedisConnectionString

Cadena de conexión a Redis (solo se usa si Provider es "Redis").

```
public string RedisConnectionString { get; set; }
```

Property Value

[string](#)

Class DatabaseConfig

Namespace: RepositorioRemoto.[Config](#)

Assembly: RepositorioRemoto.dll

Configuración de la base de datos local.

```
public class DatabaseConfig
```

Inheritance

[object](#)  ← DatabaseConfig

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Fields

SectionName

Nombre de la sección en appsettings.json.

```
public const string SectionName = "DatabaseSettings"
```

Field Value

[string](#) 

Properties

ConnectionString

Cadena de conexión a la base de datos.

```
public string ConnectionString { get; set; }
```

Property Value

[string](#)

Provider

Proveedor: "Sqlite" (perfil Dev) o "PostgreSQL" (perfil Prod).

```
public string Provider { get; set; }
```

Property Value

[string](#)

Class ExportConfig

Namespace: RepositorioRemoto.[Config](#)

Assembly: RepositorioRemoto.dll

Configuración de la exportación de usuarios a JSON.

```
public class ExportConfig
```

Inheritance

[object](#)  ← ExportConfig

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Fields

SectionName

Seccion del appsettings.json.

```
public const string SectionName = "ExportSettings"
```

Field Value

[string](#) 

Properties

FileName

Nombre del fichero exportado.

```
public string FileName { get; set; }
```

Property Value

[string](#)

OutputDirectory

Carpeta donde se guarda el fichero exportado.

```
public string OutputDirectory { get; set; }
```

Property Value

[string](#)

Class SyncConfig

Namespace: RepositorioRemoto.[Config](#)

Assembly: RepositorioRemoto.dll

Configuración de la sincronización periódica con la API remota.

```
public class SyncConfig
```

Inheritance

[object](#)  ← SyncConfig

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Fields

SectionName

Nombre de la sección en appsettings.json.

```
public const string SectionName = "SyncSettings"
```

Field Value

[string](#) 

Properties

IntervalSeconds

Segundos entre cada sincronización.

```
public int IntervalSeconds { get; set; }
```

Property Value

[int](#)

Namespace RepositorioRemoto.Dto

Classes

[AddressDto](#)

DTO de dirección.

[CompanyDto](#)

DTO de empresa.

[CreateUserRequest](#)

DTO con los datos para crear un usuario. El Id no se envía: lo asigna la API remota.

[GeoDto](#)

DTO de coordenadas.

[JsonPlaceholderUserDto](#)

DTO con la forma en que la API JSONPlaceholder devuelve un usuario.

[UpdateUserRequest](#)

DTO con los datos para actualizar un usuario existente.

[UserResponseDto](#)

DTO con los datos de un usuario que el servicio devuelve hacia fuera.

Class AddressDto

Namespace: RepositorioRemoto.[Dto](#)

Assembly: RepositorioRemoto.dll

DTO de dirección.

```
public record AddressDto : IEquatable<AddressDto>
```

Inheritance

[object](#)  ← AddressDto

Implements

[IEquatable](#)  <[AddressDto](#)>

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Extension Methods

[UserMapper.ToModel\(AddressDto\)](#)

Constructors

AddressDto(string, string, string, string, GeoDto)

DTO de dirección.

```
public AddressDto(string Street, string Suite, string City, string Zipcode, GeoDto Geo)
```

Parameters

Street [string](#) 

Calle.

Suite [string](#) 

Piso o apartamento.

City [string](#)

Ciudad.

Zipcode [string](#)

Código postal.

Geo [GeoDto](#)

Coordenadas.

Properties

City

Ciudad.

```
public string City { get; init; }
```

Property Value

[string](#)

Geo

Coordenadas.

```
public GeoDto Geo { get; init; }
```

Property Value

[GeoDto](#)

Street

Calle.

```
public string Street { get; init; }
```

Property Value

[string](#) 

Suite

Piso o apartamento.

```
public string Suite { get; init; }
```

Property Value

[string](#) 

Zipcode

Código postal.

```
public string Zipcode { get; init; }
```

Property Value

[string](#) 

Class CompanyDto

Namespace: RepositorioRemoto.[Dto](#)

Assembly: RepositorioRemoto.dll

DTO de empresa.

```
public record CompanyDto : IEquatable<CompanyDto>
```

Inheritance

[object](#)  ← CompanyDto

Implements

[IEquatable](#)  <[CompanyDto](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Extension Methods

[UserMapper.ToModel\(CompanyDto\)](#)

Constructors

CompanyDto(string, string, string)

DTO de empresa.

```
public CompanyDto(string Name, string CatchPhrase, string Bs)
```

Parameters

Name [string](#) 

Nombre de la empresa.

CatchPhrase [string](#) 

Eslogan.

Bs [string](#)

Lema de negocio.

Properties

Bs

Lema de negocio.

```
public string Bs { get; init; }
```

Property Value

[string](#)

CatchPhrase

Eslogan.

```
public string CatchPhrase { get; init; }
```

Property Value

[string](#)

Name

Nombre de la empresa.

```
public string Name { get; init; }
```

Property Value

[string](#)

Class CreateUserRequest

Namespace: RepositorioRemoto.[Dto](#)

Assembly: RepositorioRemoto.dll

DTO con los datos para crear un usuario. El Id no se envía: lo asigna la API remota.

```
public record CreateUserRequest : IEquatable<CreateUserRequest>
```

Inheritance

[object](#) ← CreateUserRequest

Implements

[IEquatable](#) <[CreateUserRequest](#)>

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Extension Methods

[UserMapper.ToUser\(CreateUserRequest, int\)](#)

Constructors

CreateUserRequest(string, string, string, AddressDto, string, string, CompanyDto)

DTO con los datos para crear un usuario. El Id no se envía: lo asigna la API remota.

```
public CreateUserRequest(string Name, string Username, string Email, AddressDto Address, string Phone, string Website, CompanyDto Company)
```

Parameters

Name [string](#)

Nombre completo.

Username [string](#)

Nombre de usuario.

Email [string](#)

Correo electrónico.

Address [AddressDto](#)

Dirección.

Phone [string](#)

Teléfono.

Website [string](#)

Página web.

Company [CompanyDto](#)

Empresa.

Properties

Address

Dirección.

```
public AddressDto Address { get; init; }
```

Property Value

[AddressDto](#)

Company

Empresa.

```
public CompanyDto Company { get; init; }
```

Property Value

[CompanyDto](#)

Email

Correo electrónico.

```
public string Email { get; init; }
```

Property Value

[string](#)

Name

Nombre completo.

```
public string Name { get; init; }
```

Property Value

[string](#)

Phone

Teléfono.

```
public string Phone { get; init; }
```

Property Value

[string](#)

Username

Nombre de usuario.

```
public string Username { get; init; }
```

Property Value

[string](#) 

Website

Página web.

```
public string Website { get; init; }
```

Property Value

[string](#) 

Class GeoDto

Namespace: RepositorioRemoto.[Dto](#)

Assembly: RepositorioRemoto.dll

DTO de coordenadas.

```
public record GeoDto : IEquatable<GeoDto>
```

Inheritance

[object](#)  ← GeoDto

Implements

[IEquatable](#)  <[GeoDto](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Extension Methods

[UserMapper.ToModel\(GeoDto\)](#)

Constructors

GeoDto(string, string)

DTO de coordenadas.

```
public GeoDto(string Lat, string Lng)
```

Parameters

Lat [string](#) 

Latitud.

Lng [string](#) 

Longitud.

Properties

Lat

Latitud.

```
public string Lat { get; init; }
```

Property Value

[string](#)

Lng

Longitud.

```
public string Lng { get; init; }
```

Property Value

[string](#)

Class JsonPlaceholderUserDto

Namespace: RepositorioRemoto.[Dto](#)

Assembly: RepositorioRemoto.dll

DTO con la forma en que la API JSONPlaceholder devuelve un usuario.

```
public record JsonPlaceholderUserDto : IEquatable<JsonPlaceholderUserDto>
```

Inheritance

[object](#)  ← JsonPlaceholderUserDto

Implements

[IEquatable](#)  <[JsonPlaceholderUserDto](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Extension Methods

[UserMapper.ToUser\(JsonPlaceholderUserDto\)](#)

Constructors

JsonPlaceholderUserDto(int, string, string, string, AddressDto, string, string, CompanyDto)

DTO con la forma en que la API JSONPlaceholder devuelve un usuario.

```
public JsonPlaceholderUserDto(int Id, string Name, string Username, string Email, AddressDto  
Address, string Phone, string Website, CompanyDto Company)
```

Parameters

Id [int](#) 

Identificador en la API.

Name [string](#) 

Nombre completo.

Username [string](#)

Nombre de usuario.

Email [string](#)

Correo electrónico.

Address [AddressDto](#)

Dirección.

Phone [string](#)

Teléfono.

Website [string](#)

Página web.

Company [CompanyDto](#)

Empresa.

Properties

Address

Dirección.

```
public AddressDto Address { get; init; }
```

Property Value

[AddressDto](#)

Company

Empresa.

```
public CompanyDto Company { get; init; }
```

Property Value

[CompanyDto](#)

Email

Correo electrónico.

```
public string Email { get; init; }
```

Property Value

[string](#)

Id

Identificador en la API.

```
public int Id { get; init; }
```

Property Value

[int](#)

Name

Nombre completo.

```
public string Name { get; init; }
```

Property Value

[string](#)

Phone

Teléfono.

```
public string Phone { get; init; }
```

Property Value

[string](#)

Username

Nombre de usuario.

```
public string Username { get; init; }
```

Property Value

[string](#)

Website

Página web.

```
public string Website { get; init; }
```

Property Value

[string](#)

Class UpdateUserRequest

Namespace: RepositorioRemoto.[Dto](#)

Assembly: RepositorioRemoto.dll

DTO con los datos para actualizar un usuario existente.

```
public record UpdateUserRequest : IEquatable<UpdateUserRequest>
```

Inheritance

[object](#)  ← UpdateUserRequest

Implements

[IEquatable](#)  <[UpdateUserRequest](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Extension Methods

[UserMapper.ToUser\(UpdateUserRequest\)](#)

Constructors

UpdateUserRequest(int, string, string, string, AddressDto, string, string, CompanyDto)

DTO con los datos para actualizar un usuario existente.

```
public UpdateUserRequest(int Id, string Name, string Username, string Email, AddressDto  
Address, string Phone, string Website, CompanyDto Company)
```

Parameters

Id [int](#) 

Identificador del usuario a actualizar.

Name [string](#) 

Nombre completo.

Username [string](#)

Nombre de usuario.

Email [string](#)

Correo electrónico.

Address [AddressDto](#)

Dirección.

Phone [string](#)

Teléfono.

Website [string](#)

Página web.

Company [CompanyDto](#)

Empresa.

Properties

Address

Dirección.

```
public AddressDto Address { get; init; }
```

Property Value

[AddressDto](#)

Company

Empresa.

```
public CompanyDto Company { get; init; }
```

Property Value

[CompanyDto](#)

Email

Correo electrónico.

```
public string Email { get; init; }
```

Property Value

[string](#)

Id

Identificador del usuario a actualizar.

```
public int Id { get; init; }
```

Property Value

[int](#)

Name

Nombre completo.

```
public string Name { get; init; }
```

Property Value

[string](#)

Phone

Teléfono.

```
public string Phone { get; init; }
```

Property Value

[string](#)

Username

Nombre de usuario.

```
public string Username { get; init; }
```

Property Value

[string](#)

Website

Página web.

```
public string Website { get; init; }
```

Property Value

[string](#)

Class UserResponseDto

Namespace: RepositorioRemoto.[Dto](#)

Assembly: RepositorioRemoto.dll

DTO con los datos de un usuario que el servicio devuelve hacia fuera.

```
public record UserResponseDto : IEquatable<UserResponseDto>
```

Inheritance

[object](#)  ← UserResponseDto

Implements

[IEquatable](#)  <[UserResponseDto](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

UserResponseDto(int, string, string, string, AddressDto, string, string, CompanyDto)

DTO con los datos de un usuario que el servicio devuelve hacia fuera.

```
public UserResponseDto(int Id, string Name, string Username, string Email, AddressDto  
Address, string Phone, string Website, CompanyDto Company)
```

Parameters

Id [int](#) 

Identificador.

Name [string](#) 

Nombre completo.

Username [string](#)

Nombre de usuario.

Email [string](#)

Correo electrónico.

Address [AddressDto](#)

Dirección.

Phone [string](#)

Teléfono.

Website [string](#)

Página web.

Company [CompanyDto](#)

Empresa.

Properties

Address

Dirección.

```
public AddressDto Address { get; init; }
```

Property Value

[AddressDto](#)

Company

Empresa.

```
public CompanyDto Company { get; init; }
```

Property Value

[CompanyDto](#)

Email

Correo electrónico.

```
public string Email { get; init; }
```

Property Value

[string](#)

Id

Identificador.

```
public int Id { get; init; }
```

Property Value

[int](#)

Name

Nombre completo.

```
public string Name { get; init; }
```

Property Value

[string](#)

Phone

Teléfono.

```
public string Phone { get; init; }
```

Property Value

[string](#) 

Username

Nombre de usuario.

```
public string Username { get; init; }
```

Property Value

[string](#) 

Website

Página web.

```
public string Website { get; init; }
```

Property Value

[string](#) 

Namespace RepositorioRemoto.Entity

Classes

[AppDbContext](#)

Contexto de Entity Framework Core para la base de datos local de usuarios. El proveedor (SQLite o PostgreSQL) se decide al registrarlo, según el perfil.

[UserEntity](#)

Entidad de persistencia para usuarios. La dirección y la empresa se guardan desmontadas en columnas de la misma tabla. La configuración (tabla, columnas y longitudes) está en AppDbContext con Fluent API.

Class AppDbContext

Namespace: RepositorioRemoto.[Entity](#)

Assembly: RepositorioRemoto.dll

Contexto de Entity Framework Core para la base de datos local de usuarios. El proveedor (SQLite o PostgreSQL) se decide al registrarlo, según el perfil.

```
public class AppDbContext : DbContext, IInfrastructure<IServiceProvider>,
    IDbContextDependencies, IDbSetCache, IDbContextPoolable, IResettableService,
    IDisposable, IAsyncDisposable
```

Inheritance

[object](#) ← [DbContext](#) ← AppDbContext

Implements

[IInfrastructure](#) <[IServiceProvider](#)>, [IDbContextDependencies](#), [IDbSetCache](#), [IDbContextPoolable](#), [IResettableService](#), [IDisposable](#), [IAsyncDisposable](#)

Inherited Members

[DbContext.Set<TEntity>\(\).](#), [DbContext.Set<TEntity>\(string\).](#),
[DbContext.OnConfiguring\(DbContextOptionsBuilder\).](#),
[DbContext.ConfigureConventions\(ModelConfigurationBuilder\).](#), [DbContext.SaveChanges\(\).](#),
[DbContext.SaveChanges\(bool\).](#), [DbContext.SaveChangesAsync\(CancellationToken\).](#),
[DbContext.SaveChangesAsync\(bool, CancellationToken\).](#), [DbContext.Dispose\(\).](#),
[DbContext.DisposeAsync\(\).](#), [DbContext.Entry<TEntity>\(TEntity\).](#), [DbContext.Entry\(object\).](#),
[DbContext.Add<TEntity>\(TEntity\).](#), [DbContext.AddAsync<TEntity>\(TEntity, CancellationToken\).](#),
[DbContext.Attach<TEntity>\(TEntity\).](#), [DbContext.Update<TEntity>\(TEntity\).](#),
[DbContext.Remove<TEntity>\(TEntity\).](#), [DbContext.Add\(object\).](#),
[DbContext.AddAsync\(object, CancellationToken\).](#), [DbContext.Attach\(object\).](#),
[DbContext.Update\(object\).](#), [DbContext.Remove\(object\).](#), [DbContext.AddRange\(params object\[\]\).](#),
[DbContext.AddRangeAsync\(params object\[\]\).](#), [DbContext.AttachRange\(params object\[\]\).](#),
[DbContext.UpdateRange\(params object\[\]\).](#), [DbContext.RemoveRange\(params object\[\]\).](#),
[DbContext.AddRange\(IEnumerable<object>\).](#),
[DbContext.AddRangeAsync\(IEnumerable<object>, CancellationToken\).](#),
[DbContext.AttachRange\(IEnumerable<object>\).](#), [DbContext.UpdateRange\(IEnumerable<object>\).](#),
[DbContext.RemoveRange\(IEnumerable<object>\).](#), [DbContext.Find\(Type, params object\[\]\).](#),
[DbContext.FindAsync\(Type, params object\[\]\).](#),
[DbContext.FindAsync\(Type, object\[\], CancellationToken\).](#), [DbContext.Find<TEntity>\(params object\[\]\).](#),
[DbContext.FindAsync<TEntity>\(params object\[\]\).](#),

[DbContext.FindAsync<TEntity>\(object\[\], CancellationToken\).↗](#) ,
[DbContext.FromExpression<TResult>\(Expression<Func<IQueryable<TResult>>>\).↗](#) ,
[DbContext.Database↗](#) , [DbContext.ChangeTracker↗](#) , [DbContext.Model↗](#) , [DbContext.ContextId↗](#) ,
[DbContext.SavingChanges↗](#) , [DbContext.SavedChanges↗](#) , [DbContext.SaveChangesFailed↗](#) ,
[object.Equals\(object\).↗](#) , [object.Equals\(object, object\).↗](#) , [object.GetHashCode\(\).↗](#) , [object.GetType\(\).↗](#) ,
[object.MemberwiseClone\(\).↗](#) , [object.ReferenceEquals\(object, object\).↗](#) , [object.ToString\(\).↗](#)

Constructors

AppDbContext(DbContextOptions<AppDbContext>)

Contexto de Entity Framework Core para la base de datos local de usuarios. El proveedor (SQLite o PostgreSQL) se decide al registrarlo, según el perfil.

```
public AppDbContext(DbContextOptions<AppDbContext> options)
```

Parameters

options [DbContextOptions↗](#) <[AppDbContext](#)>

Properties

Users

Tabla de usuarios.

```
public DbSet<UserEntity> Users { get; }
```

Property Value

[DbSet↗](#) <[UserEntity](#)>

Methods

OnModelCreating(ModelBuilder)

Override this method to further configure the model that was discovered by convention from the entity types exposed in [DbSet<TEntity>](#) properties on your derived context. The resulting model may be cached and re-used for subsequent instances of your derived context.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
```

Parameters

modelBuilder [ModelBuilder](#)

The builder being used to construct the model for this context. Databases (and other extensions) typically define extension methods on this object that allow you to configure aspects of the model that are specific to a given database.

Remarks

If a model is explicitly set on the options for this context (via [UseModel\(IModel\)](#)) then this method will not be run. However, it will still run when creating a compiled model.

See [Modeling entity types and relationships](#) for more information and examples.

Class UserEntity

Namespace: RepositorioRemoto.[Entity](#)

Assembly: RepositorioRemoto.dll

Entidad de persistencia para usuarios. La dirección y la empresa se guardan desmontadas en columnas de la misma tabla. La configuración (tabla, columnas y longitudes) está en AppDbContext con Fluent API.

```
public class UserEntity
```

Inheritance

[object](#)  ← UserEntity

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Extension Methods

[UserEntityMapper.ToUser\(UserEntity\)](#)

Properties

City

Ciudad.

```
public string City { get; set; }
```

Property Value

[string](#) 

CompanyBs

Lema de negocio de la empresa.

```
public string CompanyBs { get; set; }
```

Property Value

[string](#)

CompanyCatchPhrase

Eslogan de la empresa.

```
public string CompanyCatchPhrase { get; set; }
```

Property Value

[string](#)

CompanyName

Nombre de la empresa.

```
public string CompanyName { get; set; }
```

Property Value

[string](#)

Email

Correo electrónico.

```
public string Email { get; set; }
```

Property Value

[string](#)

Id

Identificador del usuario. Lo asigna la API remota, no la base de datos.

```
public int Id { get; set; }
```

Property Value

[int](#)

Lat

Latitud de la dirección.

```
public string Lat { get; set; }
```

Property Value

[string](#)

Lng

Longitud de la dirección.

```
public string Lng { get; set; }
```

Property Value

[string](#)

Name

Nombre completo.

```
public string Name { get; set; }
```

Property Value

[string](#)

Phone

Teléfono.

```
public string Phone { get; set; }
```

Property Value

[string](#)

Street

Calle de la dirección.

```
public string Street { get; set; }
```

Property Value

[string](#)

Suite

Piso o apartamento.

```
public string Suite { get; set; }
```

Property Value

[string](#)

Username

Nombre de usuario.

```
public string Username { get; set; }
```

Property Value

[string](#) 

Website

Página web.

```
public string Website { get; set; }
```

Property Value

[string](#) 

Zipcode

Código postal.

```
public string Zipcode { get; set; }
```

Property Value

[string](#) 

Namespace RepositorioRemoto.Errors

Classes

[DomainError](#)

Errores de dominio. Cada tipo de error construye su propio mensaje.

[DomainError.AlreadyExists](#)

Ya existe un usuario con ese identificador

[DomainError.ApiError](#)

Fallo al comunicarse con la API REST

[DomainError.NotFound](#)

cuando no existe ningún usuario con ese identificador.

[DomainError.Storage](#)

Fallo en el almacenamiento local

[DomainError.Validation](#)

Los datos no cumplen con el validador

[DomainErrors](#)

Factoría para crear errores (ver tema 15.4).

Class DomainError

Namespace: RepositorioRemoto.[Errors](#)

Assembly: RepositorioRemoto.dll

Errores de dominio. Cada tipo de error construye su propio mensaje.

```
public abstract record DomainError : IEquatable<DomainError>
```

Inheritance

[object](#)  ← DomainError

Implements

[IEquatable](#)  <[DomainError](#)>

Derived

[DomainError.AlreadyExists](#), [DomainError.ApiError](#), [DomainError.NotFound](#), [DomainError.Storage](#),
[DomainError.Validation](#)

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

DomainError(string)

Errores de dominio. Cada tipo de error construye su propio mensaje.

```
protected DomainError(string Message)
```

Parameters

Message [string](#) 

Mensaje.

Properties

Message

Mensaje.

```
public string Message { get; init; }
```

Property Value

[string](#)

Class DomainError.AlreadyExists

Namespace: RepositorioRemoto.[Errors](#)

Assembly: RepositorioRemoto.dll

Ya existe un usuario con ese identificador

```
public sealed record DomainError.AlreadyExists : DomainError, IEquatable<DomainError>,
    IEquatable<DomainError.AlreadyExists>
```

Inheritance

[object](#)  ← [DomainError](#) ← DomainError.AlreadyExists

Implements

[IEquatable](#)  <[DomainError](#)>, [IEquatable](#)  <[DomainError.AlreadyExists](#)>

Inherited Members

[DomainError.Message](#), [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

AlreadyExists(int)

Ya existe un usuario con ese identificador

```
public AlreadyExists(int Id)
```

Parameters

Id [int](#) 

Identificador

Properties

Id

Identificador

```
public int Id { get; init; }
```

Property Value

[int](#)

Class DomainError.ApiError

Namespace: RepositorioRemoto.[Errors](#)

Assembly: RepositorioRemoto.dll

Fallo al comunicarse con la API REST

```
public sealed record DomainError.ApiError : DomainError, IEquatable<DomainError>,
    IEquatable<DomainError.ApiError>
```

Inheritance

[object](#)  ← [DomainError](#) ← DomainError.ApiError

Implements

[IEquatable](#)  <[DomainError](#)>, [IEquatable](#)  <[DomainError.ApiError](#)>

Inherited Members

[DomainError.Message](#), [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

ApiError(int, string)

Fallo al comunicarse con la API REST

```
public ApiError(int StatusCode, string Detail)
```

Parameters

StatusCode [int](#) 

Código que devuelve la API.

Detail [string](#) 

Detalle del error.

Properties

Detail

Detalle del error.

```
public string Detail { get; init; }
```

Property Value

[string](#)

StatusCode

Código que devuelve la API.

```
public int StatusCode { get; init; }
```

Property Value

[int](#)

Class DomainError.NotFound

Namespace: RepositorioRemoto.[Errors](#)

Assembly: RepositorioRemoto.dll

cuando no existe ningún usuario con ese identificador.

```
public sealed record DomainError.NotFound : DomainError, IEquatable<DomainError>,
    IEquatable<DomainError.NotFound>
```

Inheritance

[object](#) < [DomainError](#) < DomainError.NotFound

Implements

[IEquatable](#) < [DomainError](#) >, [IEquatable](#) < [DomainError.NotFound](#) >

Inherited Members

[DomainError.Message](#) , [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) ,
[object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#)

Constructors

NotFound(int)

cuando no existe ningún usuario con ese identificador.

```
public NotFound(int Id)
```

Parameters

Id [int](#)

Identificador buscado.

Properties

Id

Identificador buscado.

```
public int Id { get; init; }
```

Property Value

[int](#)

Class DomainError.Storage

Namespace: RepositorioRemoto.[Errors](#)

Assembly: RepositorioRemoto.dll

Fallo en el almacenamiento local

```
public sealed record DomainError.Storage : DomainError, IEquatable<DomainError>,
    IEquatable<DomainError.Storage>
```

Inheritance

[object](#)  < [DomainError](#) < DomainError.Storage

Implements

[IEquatable](#)  <[DomainError](#)>, [IEquatable](#)  <[DomainError.Storage](#)>

Inherited Members

[DomainError.Message](#) , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

Storage(Exception)

Fallo en el almacenamiento local

```
public Storage(Exception Exception)
```

Parameters

Exception [Exception](#) 

Properties

Exception

```
public Exception Exception { get; init; }
```

Property Value

[Exception](#) 

Class DomainError.Validation

Namespace: RepositorioRemoto.[Errors](#)

Assembly: RepositorioRemoto.dll

Los datos no cumplen con el validador

```
public sealed record DomainError.Validation : DomainError, IEquatable<DomainError>,
    IEquatable<DomainError.Validation>
```

Inheritance

[object](#)  ← [DomainError](#) ← DomainError.Validation

Implements

[IEquatable](#)  <[DomainError](#)>, [IEquatable](#)  <[DomainError.Validation](#)>

Inherited Members

[DomainError.Message](#), [object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

Validation(IEnumerable<string>)

Los datos no cumplen con el validador

```
public Validation(IEnumerable<string> Errors)
```

Parameters

Errors [IEnumerable](#)  <[string](#)  >

Lista de errores detectados.

Properties

Errors

Lista de errores detectados.

```
public IEnumerable<string> Errors { get; init; }
```

Property Value

[IEnumerable](#) <[string](#)>

Class DomainErrors

Namespace: RepositorioRemoto.[Errors](#)

Assembly: RepositorioRemoto.dll

Factoría para crear errores (ver tema 15.4).

```
public static class DomainErrors
```

Inheritance

[object](#)  ← DomainErrors

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Methods

AlreadyExists(int)

usuario ya existente.

```
public static DomainError AlreadyExists(int id)
```

Parameters

id [int](#) 

Returns

[DomainError](#)

ApiError(int, string)

error de la API

```
public static DomainError ApiError(int statusCode, string detail)
```

Parameters

statusCode [int](#)↗

detail [string](#)↗

Returns

[DomainError](#)

NotFound(int)

Usuario no encontrado.

```
public static DomainError NotFound(int id)
```

Parameters

id [int](#)↗

Returns

[DomainError](#)

Storage(Exception)

Error de almacenamiento local.

```
public static DomainError Storage(Exception exception)
```

Parameters

exception [Exception](#)↗

Returns

Validation(IEnumerable<string>)

validación con la lista de errores.

```
public static DomainError Validation(IEnumerable<string> errors)
```

Parameters

errors [IEnumerable](#) <[string](#)>

Returns

[DomainError](#)

Namespace RepositorioRemoto.Infrastructure

Classes

[DependenciesProvider](#)

Centraliza el registro de dependencias de la aplicación. Patrón de la teoría 11.8.1 y de la solución 01-InyeccionDependencias.

Class DependenciesProvider

Namespace: RepositorioRemoto.[Infrastructure](#)

Assembly: RepositorioRemoto.dll

Centraliza el registro de dependencias de la aplicación. Patrón de la teoría 11.8.1 y de la solución 01-InyeccionDependencias.

```
public static class DependenciesProvider
```

Inheritance

[object](#) ← DependenciesProvider

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

BuildServiceProvider(IConfiguration)

Construye el contenedor de dependencias a partir de la configuración.

```
public static IServiceProvider BuildServiceProvider(IConfiguration configuration)
```

Parameters

configuration [IConfiguration](#)

Configuración cargada de los appsettings.

Returns

[IServiceProvider](#)

Namespace RepositorioRemoto.Mappers

Classes

[UserEntityMapper](#)

Métodos de extensión para convertir entre la entidad de persistencia y el modelo de dominio.

[UserMapper](#)

Métodos de extensión para convertir entre los DTOs y el modelo de dominio. Centraliza la conversión entre capas.

Class UserEntityMapper

Namespace: RepositorioRemoto.[Mappers](#)

Assembly: RepositorioRemoto.dll

Métodos de extensión para convertir entre la entidad de persistencia y el modelo de dominio.

```
public static class UserEntityMapper
```

Inheritance

[object](#)  ← UserEntityMapper

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

ToEntity(User)

Convierte el modelo de dominio en una fila de la tabla.

```
public static UserEntity ToEntity(this User user)
```

Parameters

user [User](#)

Returns

[UserEntity](#).

ToUser(UserEntity)

Convierte una fila de la tabla en el modelo de dominio.

```
public static User ToUser(this UserEntity entity)
```

Parameters

entity [UserEntity](#)

Returns

[User](#)

ToUser(IEnumerable<UserEntity>)

Convierte una colección de filas en una lista de usuarios.

```
public static List<User> ToUser(this IEnumerable<UserEntity> entities)
```

Parameters

entities [IEnumerable](#) <[UserEntity](#)>

Returns

[List](#) <[User](#)>

Class UserMapper

Namespace: RepositorioRemoto.[Mappers](#)

Assembly: RepositorioRemoto.dll

Métodos de extensión para convertir entre los DTOs y el modelo de dominio. Centraliza la conversión entre capas.

```
public static class UserMapper
```

Inheritance

[object](#)  ← UserMapper

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Methods

ToCreateRequest(User)

Convierte un User en la petición de creación que se envía a la API.

```
public static CreateUserRequest ToCreateRequest(this User user)
```

Parameters

user [User](#)

Returns

[CreateUserRequest](#)

ToDto(Address)

Convierte la dirección del modelo en su DTO.

```
public static AddressDto ToDto(this Address address)
```

Parameters

address [Address](#)

Returns

[AddressDto](#)

ToDto(Company)

Convierte la empresa del modelo en su DTO.

```
public static CompanyDto ToDto(this Company company)
```

Parameters

company [Company](#)

Returns

[CompanyDto](#)

ToDto(Geo)

Convierte las coordenadas del modelo en su DTO.

```
public static GeoDto ToDto(this Geo geo)
```

Parameters

geo [Geo](#)

Returns

[GeoDto](#)

ToModel(AddressDto)

Convierte el DTO de dirección en el modelo.

```
public static Address ToModel(this AddressDto dto)
```

Parameters

dto [AddressDto](#)

Returns

[Address](#)

ToModel(CompanyDto)

Convierte el DTO de empresa en el modelo.

```
public static Company ToModel(this CompanyDto dto)
```

Parameters

dto [CompanyDto](#)

Returns

[Company](#)

ToModel(GeoDto)

Convierte el DTO de coordenadas en el modelo.

```
public static Geo ToModel(this GeoDto dto)
```

Parameters

dto [GeoDto](#)

Returns

[Geo](#)

ToResponse(User)

Convierte un User en el DTO de respuesta.

```
public static UserResponseDto ToResponse(this User user)
```

Parameters

user [User](#)

Returns

[UserResponseDto](#)

ToUpdateRequest(User)

Convierte un User en la petición de actualización que se envía a la API.

```
public static UpdateUserRequest ToUpdateRequest(this User user)
```

Parameters

user [User](#)

Returns

[UpdateUserRequest](#)

ToUser(CreateUserRequest, int)

Convierte una petición de creación en User. El id lo asigna la API.

```
public static User ToUser(this CreateUserRequest request, int id = 0)
```

Parameters

request [CreateUserRequest](#)

id [int](#) 

Returns

[User](#)

ToUser(JsonPlaceholderUserDto)

Convierte el usuario recibido de la API en el modelo de dominio.

```
public static User ToUser(this JsonPlaceholderUserDto dto)
```

Parameters

dto [JsonPlaceholderUserDto](#)

Returns

[User](#)

ToUser(UpdateUserRequest)

Convierte una petición de actualización en User.

```
public static User ToUser(this UpdateUserRequest request)
```

Parameters

request [UpdateUserRequest](#)

Returns

Namespace RepositorioRemoto.Models

Classes

[Address](#)

Direccion de le usuario

[Company](#)

Empresa en la que trabaja cada usuario

[Geo](#)

Coordenadas de la vivienda del usuario

[User](#)

Modelo de dominio que representa un usuario.

Class Address

Namespace: RepositorioRemoto.[Models](#)

Assembly: RepositorioRemoto.dll

Direccion de le usuario

```
public record Address : IEquatable<Address>
```

Inheritance

[object](#) ← Address

Implements

[IEquatable](#) <[Address](#)>

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Extension Methods

[UserMapper.ToDto\(Address\)](#)

Constructors

Address(string, string, string, string, Geo)

Direccion de le usuario

```
public Address(string Street, string Suite, string City, string Zipcode, Geo Geo)
```

Parameters

Street [string](#)

calle

Suite [string](#)

psio

City [string](#)

Ciudad

Zipcode [string](#)

Codigo postal

Geo [Geo](#)

Cordenadas de direccion

Properties

City

Ciudad

```
public string City { get; init; }
```

Property Value

[string](#)

Geo

Cordenadas de direccion

```
public Geo Geo { get; init; }
```

Property Value

[Geo](#)

Street

calle

```
public string Street { get; init; }
```

Property Value

[string](#) 

Suite

psio

```
public string Suite { get; init; }
```

Property Value

[string](#) 

Zipcode

Codigo postal

```
public string Zipcode { get; init; }
```

Property Value

[string](#) 

Class Company

Namespace: RepositorioRemoto.[Models](#)

Assembly: RepositorioRemoto.dll

Empresa en la que trabaja cada usuario

```
public record Company : IEquatable<Company>
```

Inheritance

[object](#) ← Company

Implements

[IEquatable](#) <[Company](#)>

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Extension Methods

[UserMapper.ToDto\(Company\)](#)

Constructors

Company(string, string, string)

Empresa en la que trabaja cada usuario

```
public Company(string Name, string CatchPhrase, string Bs)
```

Parameters

Name [string](#)

Nombre de la empresa

CatchPhrase [string](#)

Eslogan

Bs [string](#)

Lema de la empresa

Properties

Bs

Lema de la empresa

```
public string Bs { get; init; }
```

Property Value

[string](#)

CatchPhrase

Eslogan

```
public string CatchPhrase { get; init; }
```

Property Value

[string](#)

Name

Nombre de la empresa

```
public string Name { get; init; }
```

Property Value

[string](#)

Class Geo

Namespace: RepositorioRemoto.[Models](#)

Assembly: RepositorioRemoto.dll

Coordenadas de la vivienda del usuario

```
public record Geo : IEquatable<Geo>
```

Inheritance

[object](#) ← Geo

Implements

[IEquatable](#) <[Geo](#)>

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Extension Methods

[UserMapper.ToDto\(Geo\)](#)

Constructors

Geo(string, string)

Coordenadas de la vivienda del usuario

```
public Geo(string Lat, string Lng)
```

Parameters

Lat [string](#)

Latitud

Lng [string](#)

Longitud

Properties

Lat

Latitud

```
public string Lat { get; init; }
```

Property Value

[string](#)

Lng

Longitud

```
public string Lng { get; init; }
```

Property Value

[string](#)

Class User

Namespace: RepositorioRemoto.[Models](#)

Assembly: RepositorioRemoto.dll

Modelo de dominio que representa un usuario.

```
public record User : IEquatable<User>
```

Inheritance

[object](#)  ← User

Implements

[IEquatable](#)  <[User](#)>

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  ,
[object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Extension Methods

[UserEntityMapper.ToEntity\(User\)](#) , [UserMapper.ToCreateRequest\(User\)](#) , [UserMapper.ToResponse\(User\)](#) ,
[UserMapper.ToUpdateRequest\(User\)](#).

Constructors

User(int, string, string, string, Address, string, string, Company)

Modelo de dominio que representa un usuario.

```
public User(int Id, string Name, string Username, string Email, Address Address, string  
Phone, string Website, Company Company)
```

Parameters

Id [int](#) 

Id

Name [string](#) 

Nombre

Username [string](#)

Nombre de usuario

Email [string](#)

Direccion email

Address [Address](#)

Residencia

Phone [string](#)

Numero de telefono

Website [string](#)

pagina web

Company [Company](#)

Empresa para la quie trabaja

Properties

Address

Residencia

```
public Address Address { get; init; }
```

Property Value

[Address](#)

Company

Empresa para la quie trabaja

```
public Company Company { get; init; }
```

Property Value

[Company](#)

Email

Direccion email

```
public string Email { get; init; }
```

Property Value

[string](#)

Id

Id

```
public int Id { get; init; }
```

Property Value

[int](#)

Name

Nombre

```
public string Name { get; init; }
```

Property Value

[string](#)

Phone

Numero de telefono

```
public string Phone { get; init; }
```

Property Value

[string](#)

Username

Nombre de ususario

```
public string Username { get; init; }
```

Property Value

[string](#)

Website

pagina web

```
public string Website { get; init; }
```

Property Value

[string](#)

Namespace RepositorioRemoto.Notifications

Classes

[ConsoleNotificationService](#)

Servicio de notificaciones con Rx.NET. Observable CALIENTE: quien se suscribe tarde no ve los eventos anteriores. Patrón tomado de la solución 14-ReactividadRxNet (SensorService).

[Notification](#)

Evento que viaja por el flujo de notificaciones.

Interfaces

[INotificationService](#)

Contrato del servicio de notificaciones. Emite un flujo de eventos al que Program.cs se suscribe al arrancar. Va inyectado en el UserService, que lo llama al crear, actualizar y eliminar.

Enums

[NotificationType](#)

Tipo de evento que se notifica.

Class ConsoleNotificationService

Namespace: RepositorioRemoto.[Notifications](#)

Assembly: RepositorioRemoto.dll

Servicio de notificaciones con Rx.NET. Observable CALIENTE: quien se suscribe tarde no ve los eventos anteriores. Patrón tomado de la solución 14-ReactividadRxNet (SensorService).

```
public class ConsoleNotificationService : INotificationService, IDisposable
```

Inheritance

[object](#) ← ConsoleNotificationService

Implements

[INotificationService](#), [IDisposable](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Properties

Notifications

Flujo de notificaciones al que suscribirse.

```
public IObservable<Notification> Notifications { get; }
```

Property Value

[IObservable](#) <[Notification](#)>

Methods

Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public void Dispose()
```

NotifyCreated(User)

Notifica que se ha creado un usuario.

```
public void NotifyCreated(User user)
```

Parameters

user [User](#)

Usuario creado.

NotifyDeleted(int)

Notifica que se ha eliminado un usuario.

```
public void NotifyDeleted(int id)
```

Parameters

id [int](#)

Identificador del usuario eliminado.

NotifyUpdated(User)

Notifica que se ha actualizado un usuario.

```
public void NotifyUpdated(User user)
```

Parameters

user [User](#)

Usuario actualizado.

Interface INotificationService

Namespace: RepositorioRemoto.[Notifications](#)

Assembly: RepositorioRemoto.dll

Contrato del servicio de notificaciones. Emite un flujo de eventos al que Program.cs se suscribe al arrancar. Va inyectado en el UserService, que lo llama al crear, actualizar y eliminar.

```
public interface INotificationService
```

Properties

Notifications

Flujo de notificaciones al que suscribirse.

```
IObservable<Notification> Notifications { get; }
```

Property Value

[IObservable](#) <[Notification](#)>

Methods

NotifyCreated(User)

Notifica que se ha creado un usuario.

```
void NotifyCreated(User user)
```

Parameters

user [User](#)

Usuario creado.

NotifyDeleted(int)

Notifica que se ha eliminado un usuario.

```
void NotifyDeleted(int id)
```

Parameters

id [int](#)

Identificador del usuario eliminado.

NotifyUpdated(User)

Notifica que se ha actualizado un usuario.

```
void NotifyUpdated(User user)
```

Parameters

user [User](#)

Usuario actualizado.

Class Notification

Namespace: RepositorioRemoto.[Notifications](#)

Assembly: RepositorioRemoto.dll

Evento que viaja por el flujo de notificaciones.

```
public record Notification : IEquatable<Notification>
```

Inheritance

[object](#)  ← Notification

Implements

[IEquatable](#)  <[Notification](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) ,
[object.MemberwiseClone\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

Notification(NotificationType, string, DateTime)

Evento que viaja por el flujo de notificaciones.

```
public Notification(NotificationType Type, string Message, DateTime Timestamp)
```

Parameters

Type [NotificationType](#)

Tipo de evento.

Message [string](#) 

Mensaje legible para mostrar.

Timestamp [DateTime](#) 

Momento en que se produjo.

Properties

Message

Mensaje legible para mostrar.

```
public string Message { get; init; }
```

Property Value

[string](#)

Timestamp

Momento en que se produjo.

```
public DateTime Timestamp { get; init; }
```

Property Value

[DateTime](#)

Type

Tipo de evento.

```
public NotificationType Type { get; init; }
```

Property Value

[NotificationType](#)

Enum NotificationType

Namespace: RepositorioRemoto.[Notifications](#)

Assembly: RepositorioRemoto.dll

Tipo de evento que se notifica.

```
public enum NotificationType
```

Fields

Create = 0

Se ha creado un usuario.

Delete = 2

Se ha eliminado un usuario.

Update = 1

Se ha actualizado un usuario.

Namespace RepositorioRemoto.Repositories

Classes

[UserRemoteRepository](#)

Repositorio remoto sobre el cliente Refit de JSONPlaceholder. Traduce los fallos HTTP en DomainError para que el resto de la aplicación no trabaje con excepciones (patrón de la solución 07-Refit, tema 15).

[UserRepository](#)

Repositorio local de usuarios con Entity Framework Core. Funciona igual con SQLite (perfil Dev) y con PostgreSQL (perfil Prod): el proveedor se elige al registrar el AppDbContext.

Interfaces

[IUserRemoteRepository](#)

Contrato del repositorio remoto: habla con la API REST de JSONPlaceholder. Envuelve al cliente Refit y traduce los fallos HTTP en DomainError.ApiError, para que el resto de la aplicación no trabaje con excepciones.

[IUserRepository](#)

Contrato del repositorio local de usuarios. Devuelve valores planos: null o false cuando no se encuentra algo. Es el servicio quien los convierte en Result.

Interface IUserRemoteRepository

Namespace: RepositorioRemoto.[Repositories](#)

Assembly: RepositorioRemoto.dll

Contrato del repositorio remoto: habla con la API REST de JSONPlaceholder. Envuelve al cliente Refit y traduce los fallos HTTP en DomainError.ApiError, para que el resto de la aplicación no trabaje con excepciones.

```
public interface IUserRemoteRepository
```

Methods

CreateAsync(CreateUserRequest)

Crea un usuario en la API. Devuelve el usuario con el id que asigna la API.

```
Task<Result<User, DomainError>> CreateAsync(CreateUserRequest request)
```

Parameters

request [CreateUserRequest](#)

Datos del usuario a crear.

Returns

[Task](#) [<Result<\[User\]\(#\), \[DomainError\]\(#\)>>](#)

DeleteAsync(int)

Elimina un usuario de la API.

```
Task<Result<bool, DomainError>> DeleteAsync(int id)
```

Parameters

id [int](#)

Identificador del usuario a eliminar.

Returns

[Task](#) <Result<[bool](#), [DomainError](#)>>

GetAllAsync()

Obtiene todos los usuarios de la API.

```
Task<Result<List<User>, DomainError>> GetAllAsync()
```

Returns

[Task](#) <Result<[List](#) <[User](#)>, [DomainError](#)>>

GetByIdAsync(int)

Obtiene un usuario por su identificador. NotFound si la API no lo tiene.

```
Task<Result<User, DomainError>> GetByIdAsync(int id)
```

Parameters

id [int](#)

Identificador del usuario.

Returns

[Task](#) <Result<[User](#), [DomainError](#)>>

UpdateAsync(int, UpdateUserRequest)

Actualiza un usuario en la API.

```
Task<Result<User, DomainError>> UpdateAsync(int id, UpdateUserRequest request)
```

Parameters

id [int](#)

Identificador del usuario a actualizar.

request [UpdateUserRequest](#)

Datos nuevos.

Returns

[Task](#) <Result<[User](#), [DomainError](#)> >

Interface IUserRepository

Namespace: RepositorioRemoto.[Repositories](#)

Assembly: RepositorioRemoto.dll

Contrato del repositorio local de usuarios. Devuelve valores planos: null o false cuando no se encuentra algo. Es el servicio quien los convierte en Result.

```
public interface IUserRepository
```

Methods

CreateAllAsync(IEnumerable<User>)

Guarda varios usuarios de una vez. Lo usa la sincronización.

```
Task CreateAllAsync(IEnumerable<User> users)
```

Parameters

users [IEnumerable](#) [User](#)

Returns

[Task](#)

CreateAsync(User)

Guarda un usuario nuevo.

```
Task<User> CreateAsync(User user)
```

Parameters

user [User](#)

Returns

[Task](#) <[User](#)>

DeleteAllAsync()

Vacía la tabla. Lo usan el arranque y la sincronización.

```
Task DeleteAllAsync()
```

Returns

[Task](#)

DeleteAsync(int)

Elimina un usuario. Devuelve false si no existía.

```
Task<bool> DeleteAsync(int id)
```

Parameters

id [int](#)

Returns

[Task](#) <[bool](#)>

GetAllAsync()

Obtiene todos los usuarios guardados.

```
Task<IEnumerable<User>> GetAllAsync()
```

Returns

[Task](#) <[IEnumerable](#) <[User](#)>>

GetByIdAsync(int)

Obtiene un usuario por su identificador, o null si no existe.

```
Task<User?> GetByIdAsync(int id)
```

Parameters

id [int](#)

Returns

[Task](#) <[User](#)>

UpdateAsync(User)

Actualiza un usuario existente, o devuelve null si no existe.

```
Task<User?> UpdateAsync(User user)
```

Parameters

user [User](#)

Returns

[Task](#) <[User](#)>

Class UserRemoteRepository

Namespace: RepositorioRemoto.[Repositories](#)

Assembly: RepositorioRemoto.dll

Repositorio remoto sobre el cliente Refit de JSONPlaceholder. Traduce los fallos HTTP en DomainError para que el resto de la aplicación no trabaje con excepciones (patrón de la solución 07-Refit, tema 15).

```
public class UserRemoteRepository : IUserRemoteRepository
```

Inheritance

[object](#) ← UserRemoteRepository

Implements

[IUserRemoteRepository](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

UserRemoteRepository(IJsonPlaceholderApi)

Repositorio remoto sobre el cliente Refit de JSONPlaceholder. Traduce los fallos HTTP en DomainError para que el resto de la aplicación no trabaje con excepciones (patrón de la solución 07-Refit, tema 15).

```
public UserRemoteRepository(IJsonPlaceholderApi api)
```

Parameters

api [IJsonPlaceholderApi](#)

Methods

CreateAsync(CreateUserRequest)

Crea un usuario en la API. Devuelve el usuario con el id que asigna la API.

```
public Task<Result<User, DomainError>> CreateAsync(CreateUserRequest request)
```

Parameters

request [CreateUserRequest](#)

Datos del usuario a crear.

Returns

[Task](#) [<Result<\[User\]\(#\), \[DomainError\]\(#\)>>](#)

DeleteAsync(int)

Elimina un usuario de la API.

```
public Task<Result<bool, DomainError>> DeleteAsync(int id)
```

Parameters

id [int](#)

Identificador del usuario a eliminar.

Returns

[Task](#) [<Result<\[bool\]\(#\), \[DomainError\]\(#\)>>](#)

GetAllAsync()

Obtiene todos los usuarios de la API.

```
public Task<Result<List<User>, DomainError>> GetAllAsync()
```

Returns

[Task](#) <Result<[List](#) <[User](#)>, [DomainError](#)>>

GetByIdAsync(int)

Obtiene un usuario por su identificador. NotFound si la API no lo tiene.

```
public Task<Result<User, DomainError>> GetByIdAsync(int id)
```

Parameters

id [int](#)

Identificador del usuario.

Returns

[Task](#) <Result<[User](#), [DomainError](#)>>

UpdateAsync(int, UpdateUserRequest)

Actualiza un usuario en la API.

```
public Task<Result<User, DomainError>> UpdateAsync(int id, UpdateUserRequest request)
```

Parameters

id [int](#)

Identificador del usuario a actualizar.

request [UpdateUserRequest](#)

Datos nuevos.

Returns

[Task](#) <Result<[User](#), [DomainError](#)>>

Class UserRepository

Namespace: RepositorioRemoto.[Repositories](#)

Assembly: RepositorioRemoto.dll

Repositorio local de usuarios con Entity Framework Core. Funciona igual con SQLite (perfil Dev) y con PostgreSQL (perfil Prod): el proveedor se elige al registrar el ApplicationDbContext.

```
public class UserRepository : IUserRepository
```

Inheritance

[object](#) ← UserRepository

Implements

[IUserRepository](#).

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

UserRepository(AppDbContext)

Repositorio local de usuarios con Entity Framework Core. Funciona igual con SQLite (perfil Dev) y con PostgreSQL (perfil Prod): el proveedor se elige al registrar el ApplicationDbContext.

```
public UserRepository(AppDbContext context)
```

Parameters

context [AppDbContext](#)

Methods

CreateAllAsync(IEnumerable<User>)

Guarda varios usuarios de una vez. Lo usa la sincronización.

```
public Task CreateAllAsync(IEnumerable<User> users)
```

Parameters

users [IEnumerable](#) <[User](#)>

Returns

[Task](#)

CreateAsync(User)

Crea un nuevo usuario.

```
public Task<User> CreateAsync(User user)
```

Parameters

user [User](#)

Usuario que se va a crear.

Returns

[Task](#) <[User](#)>

El usuario creado.

DeleteAllAsync()

Elimina todos los usuarios almacenados.

```
public Task DeleteAllAsync()
```

Returns

[Task](#)

DeleteAsync(int)

Elimina un usuario por su identificador.

```
public Task<bool> DeleteAsync(int id)
```

Parameters

id [int](#)

Identificador del usuario que se quiere eliminar.

Returns

[Task](#) <[bool](#)>

True si se elimina correctamente; false si el usuario no existe.

GetAllAsync()

Obtiene todos los usuarios.

```
public Task<IEnumerable<User>> GetAllAsync()
```

Returns

[Task](#) <[IEnumerable](#) <[User](#)> >

Una colección con todos los usuarios.

GetByIdAsync(int)

Obtiene un usuario por su identificador.

```
public Task<User?> GetByIdAsync(int id)
```

Parameters

id [int](#)

Identificador del usuario.

Returns

[Task](#) <[User](#)>

El usuario encontrado o null si no existe.

UpdateAsync(User)

Actualiza los datos de un usuario existente.

```
public Task<User?> UpdateAsync(User user)
```

Parameters

user [User](#)

Usuario con los datos actualizados.

Returns

[Task](#) <[User](#)>

El usuario actualizado o null si no existe.

Namespace RepositorioRemoto.Services

Classes

[UserService](#)

Servicio de usuarios. Orquesta los tres niveles de almacenamiento (caché, base de datos local y API REST) y notifica las escrituras.

Interfaces

[IUserService](#)

Contrato del servicio de usuarios: orquesta los tres niveles de almacenamiento (caché, base de datos local y API REST remota) y notifica las escrituras.

Interface IUserService

Namespace: RepositorioRemoto.[Services](#)

Assembly: RepositorioRemoto.dll

Contrato del servicio de usuarios: orquesta los tres niveles de almacenamiento (caché, base de datos local y API REST remota) y notifica las escrituras.

```
public interface IUserService
```

Methods

CreateAsync(CreateUserRequest)

Crea un usuario: lo valida, lo envía a la API remota y guarda el resultado en la base de datos local y en la caché.

```
Task<Result<UserResponseDto, DomainError>> CreateAsync(CreateUserRequest request)
```

Parameters

request [CreateUserRequest](#)

Datos del usuario a crear.

Returns

[Task](#) [<Result<\[UserResponseDto\]\(#\), \[DomainError\]\(#\)>>](#)

DeleteAsync(int)

Elimina un usuario de la API remota, de la base de datos local y de la caché.

```
Task<Result<bool, DomainError>> DeleteAsync(int id)
```

Parameters

`id` [int](#)

Identificador del usuario a eliminar.

Returns

[Task](#) <Result<[bool](#), [DomainError](#)>>

ExportToJsonAsync()

Exporta todos los usuarios a un fichero JSON.

```
Task<Result<string, DomainError>> ExportToJsonAsync()
```

Returns

[Task](#) <Result<[string](#), [DomainError](#)>>

La ruta del fichero generado.

GetAllAsync()

Obtiene todos los usuarios de la base de datos local. Si está vacía, los trae de la API remota y los guarda antes de devolverlos.

```
Task<Result<List<UserResponseDto>, DomainError>> GetAllAsync()
```

Returns

[Task](#) <Result<[List](#) <[UserResponseDto](#)>, [DomainError](#)>>

GetByIdAsync(int)

Obtiene un usuario buscando en caché, luego en la base de datos local y por último en la API remota. NotFound si no está en ninguno.

```
Task<Result<UserResponseDto, DomainError>> GetByIdAsync(int id)
```

Parameters

id [int](#)

Identificador del usuario.

Returns

[Task](#) <Result<[UserResponseDto](#), [DomainError](#)>>

UpdateAsync(int, UpdateUserRequest)

Actualiza un usuario: lo valida, lo envía a la API remota y refresca la base de datos local y la caché.

```
Task<Result<UserResponseDto, DomainError>> UpdateAsync(int id, UpdateUserRequest request)
```

Parameters

id [int](#)

Identificador del usuario a actualizar.

request [UpdateUserRequest](#)

Datos nuevos.

Returns

[Task](#) <Result<[UserResponseDto](#), [DomainError](#)>>

Class UserService

Namespace: RepositorioRemoto.[Services](#)

Assembly: RepositorioRemoto.dll

Servicio de usuarios. Orquesta los tres niveles de almacenamiento (caché, base de datos local y API REST) y notifica las escrituras.

```
public class UserService : IUserService
```

Inheritance

[object](#) ← UserService

Implements

[IUserService](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

UserService(IUserRepository, IUserRemoteRepository, ICacheService, IUserValidator, INotificationService, IOptions<ExportConfig>, ILogger<UserService>)

Servicio de usuarios. Orquesta los tres niveles de almacenamiento (caché, base de datos local y API REST) y notifica las escrituras.

```
public UserService(IUserRepository localRepository, IUserRemoteRepository remoteRepository,
    ICacheService cache, IUserValidator validator, INotificationService notificationService,
    IOptions<ExportConfig> exportOptions, ILogger<UserService> logger)
```

Parameters

localRepository [IUserRepository](#)

remoteRepository [IUserRemoteRepository](#)

cache [ICacheService](#)

validator [IValidator](#)

notificationService [INotificationService](#)

exportOptions [IOptions](#) <[ExportConfig](#)>

logger [ILogger](#) <[UserService](#)>

Methods

CreateAsync(CreateUserRequest)

Crea un usuario: lo valida, lo envía a la API remota y guarda el resultado en la base de datos local y en la caché.

```
public Task<Result<UserResponseDto, DomainError>> CreateAsync(CreateUserRequest request)
```

Parameters

request [CreateUserRequest](#)

Datos del usuario a crear.

Returns

[Task](#) <[Result](#) <[UserResponseDto](#), [DomainError](#)>>

DeleteAsync(int)

Elimina un usuario de la API remota, de la base de datos local y de la caché.

```
public Task<Result<bool, DomainError>> DeleteAsync(int id)
```

Parameters

id [int](#)

Identificador del usuario a eliminar.

Returns

[Task](#) <Result<[bool](#), [DomainError](#)>>

ExportToJsonAsync()

Exporta todos los usuarios a un fichero JSON.

```
public Task<Result<string, DomainError>> ExportToJsonAsync()
```

Returns

[Task](#) <Result<[string](#), [DomainError](#)>>

La ruta del fichero generado.

GetAllAsync()

Obtiene todos los usuarios de la base de datos local. Si está vacía, los trae de la API remota y los guarda antes de devolverlos.

```
public Task<Result<List<UserResponseDto>, DomainError>> GetAllAsync()
```

Returns

[Task](#) <Result<[List](#) <[UserResponseDto](#)>, [DomainError](#)>>

GetByIdAsync(int)

Obtiene un usuario buscando en caché, luego en la base de datos local y por último en la API remota. NotFound si no está en ninguno.

```
public Task<Result<UserResponseDto, DomainError>> GetByIdAsync(int id)
```

Parameters

id [int](#)

Identificador del usuario.

Returns

[Task](#) <Result<[UserResponseDto](#), [DomainError](#)>>

UpdateAsync(int, UpdateUserRequest)

Actualiza un usuario: lo valida, lo envía a la API remota y refresca la base de datos local y la caché.

```
public Task<Result<UserResponseDto, DomainError>> UpdateAsync(int id,
UpdateUserRequest request)
```

Parameters

id [int](#)

Identificador del usuario a actualizar.

request [UpdateUserRequest](#)

Datos nuevos.

Returns

[Task](#) <Result<[UserResponseDto](#), [DomainError](#)>>

Namespace RepositorioRemoto.Sync

Classes

[UserSyncBackgroundService](#)

Tarea en segundo plano que lanza la sincronización cada X segundos. Solo se ocupa del reloj: la lógica está en UserSyncService.

[UserSyncService](#)

Sincroniza el almacenamiento local con la API remota. Sigue el orden del enunciado: vaciar caché, vaciar BD, pedir todos y guardarlos.

Interfaces

[IUserSyncService](#)

Contrato de la sincronización entre la API remota y el almacenamiento local. Se separa del BackgroundService para poder testear la lógica con mocks (buenas prácticas 22.9).

Interface IUserSyncService

Namespace: RepositorioRemoto.[Sync](#)

Assembly: RepositorioRemoto.dll

Contrato de la sincronización entre la API remota y el almacenamiento local. Se separa del BackgroundService para poder testear la lógica con mocks (buenas prácticas 22.9).

```
public interface IUserSyncService
```

Methods

SyncAsync()

Vacía la caché y la base de datos local y las recarga con lo que haya en la API.

```
Task<bool> SyncAsync()
```

Returns

[Task](#) <[bool](#)>

true si la sincronización se completó; false si falló la API.

Class UserSyncBackgroundService

Namespace: RepositorioRemoto.[Sync](#)

Assembly: RepositorioRemoto.dll

Tarea en segundo plano que lanza la sincronización cada X segundos. Solo se ocupa del reloj: la lógica está en UserSyncService.

```
public class UserSyncBackgroundService : BackgroundService, IHostedService, IDisposable
```

Inheritance

[object](#) ← [BackgroundService](#) ← UserSyncBackgroundService

Implements

[IHostedService](#), [IDisposable](#)

Inherited Members

[BackgroundService.StartAsync\(CancellationToken\)](#),
[BackgroundService.StopAsync\(CancellationToken\)](#), [BackgroundService.Dispose\(\)](#),
[BackgroundService.ExecuteTask](#), [object.Equals\(object\)](#), [object.Equals\(object, object\)](#),
[object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#),
[object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

UserSyncBackgroundService(IServiceProvider,
IOptions<SyncConfig>, ILogger<UserSyncBackgroundService>)

Tarea en segundo plano que lanza la sincronización cada X segundos. Solo se ocupa del reloj: la lógica está en UserSyncService.

```
public UserSyncBackgroundService(IServiceProvider serviceProvider, IOptions<SyncConfig>  
config, ILogger<UserSyncBackgroundService> logger)
```

Parameters

serviceProvider [IServiceProvider](#)

`config` [IOptions](#) <[SyncConfig](#)>

`logger` [ILogger](#) <[UserSyncBackgroundService](#)>

Methods

ExecuteAsync(CancellationToken)

This method is called when the [IHostedService](#) starts. The implementation should return a task that represents the lifetime of the long running operation(s) being performed.

```
protected override Task ExecuteAsync(CancellationToken stoppingToken)
```

Parameters

`stoppingToken` [CancellationToken](#)

Triggered when [StopAsync\(CancellationToken\)](#) is called.

Returns

[Task](#)

A [Task](#) that represents the long running operations.

Remarks

See [Worker Services in .NET](#) for implementation guidelines.

Class UserSyncService

Namespace: RepositorioRemoto.[Sync](#)

Assembly: RepositorioRemoto.dll

Sincroniza el almacenamiento local con la API remota. Sigue el orden del enunciado: vaciar caché, vaciar BD, pedir todos y guardarlos.

```
public class UserSyncService : IUserSyncService
```

Inheritance

[object](#) ← UserSyncService

Implements

[IUserSyncService](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Constructors

UserSyncService(IUserRemoteRepository, IUserRepository, ICacheService, ILogger<UserSyncService>)

Sincroniza el almacenamiento local con la API remota. Sigue el orden del enunciado: vaciar caché, vaciar BD, pedir todos y guardarlos.

```
public UserSyncService(IUserRemoteRepository remoteRepository, IUserRepository  
    localRepository, ICacheService cache, ILogger<UserSyncService> logger)
```

Parameters

remoteRepository [IUserRemoteRepository](#)

localRepository [IUserRepository](#)

cache [ICacheService](#)

logger [ILogger](#) <[UserSyncService](#)>

Methods

SyncAsync()

Vacía la caché y la base de datos local y las recarga con lo que haya en la API.

```
public Task<bool> SyncAsync()
```

Returns

[Task](#) <[bool](#)>

true si la sincronización se completó; false si falló la API.

Namespace RepositorioRemoto.Validators

Classes

[UserValidator](#)

Valida los datos de usuario antes de enviarlos a la API remota. Devuelve todos los fallos encontrados.

Interfaces

[IUserValidator](#)

Interfaz del validador de peticiones de usuario. Aunque el repo aconseja no crear interfaces por crear, la creamos para poder testear despues

Interface IUserValidator

Namespace: RepositorioRemoto.[Validators](#)

Assembly: RepositorioRemoto.dll

Interfaz del validador de peticiones de usuario. Aunque el repo aconseja no crear interfaces por crear, la creamos para poder testear despues

```
public interface IUserValidator
```

Methods

ValidateCreate(CreateUserRequest)

Comprueba que los datos de creación son correctos.

```
Result<CreateUserRequest, DomainError> ValidateCreate(CreateUserRequest request)
```

Parameters

request [CreateUserRequest](#)

Datos recibidos para crear el usuario.

Returns

Result<[CreateUserRequest](#), [DomainError](#)>

petición válida o error de dominio

ValidateUpdate(UpdateUserRequest)

Comprueba que los datos de actualización son correctos.

```
Result<UpdateUserRequest, DomainError> ValidateUpdate(UpdateUserRequest request)
```

Parameters

request [UpdateUserRequest](#)

Datos para actualizar el usuario.

Returns

Result<[UpdateUserRequest](#), [DomainError](#)>

Peticion o un error de dominio.

Class UserValidator

Namespace: RepositorioRemoto.[Validators](#)

Assembly: RepositorioRemoto.dll

Valida los datos de usuario antes de enviarlos a la API remota. Devuelve todos los fallos encontrados.

```
public class UserValidator : IUserValidator
```

Inheritance

[object](#) ← UserValidator

Implements

[IUserValidator](#)

Inherited Members

[object.Equals\(object\)](#), [object.Equals\(object, object\)](#), [object.GetHashCode\(\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#), [object.ToString\(\)](#)

Methods

ValidateCreate(CreateUserRequest)

Valida los datos necesarios para crear un usuario.

```
public Result<CreateUserRequest, DomainError> ValidateCreate(CreateUserRequest request)
```

Parameters

request [CreateUserRequest](#)

Datos del usuario que se va a crear.

Returns

Result<[CreateUserRequest](#), [DomainError](#)>

El resultado de la validación, con el usuario si es correcto o un error de dominio si hay fallos.

ValidateUpdate(UpdateUserRequest)

Valida los datos necesarios para actualizar un usuario.

```
public Result<UpdateUserRequest, DomainError> ValidateUpdate(UpdateUserRequest request)
```

Parameters

request [UpdateUserRequest](#)

Datos del usuario que se va a actualizar.

Returns

Result<[UpdateUserRequest](#), [DomainError](#)>

El resultado de la validación, con el usuario si es correcto o un error de dominio si hay fallos.